Table des matières

Cours 09 — Ingérer un flux d'événements 1
Objectifs 2
1. Ce que le temps réel apporte, et ce qu'il coûte 3
2. MQTT : un courtier, des éditeurs, des abonnés 4
3. Les sujets, et leurs jokers 4
4. La qualité de service, et ce que « livrer » veut dire 5
5. Le message retenu, et la session persistante 6
6. L'archive, c'est ce qu'écrit un abonné 6
7. Kafka : un journal qu'on garde 7
8. Partitions, clés et ordre 8
9. Groupes de consommateurs, décalages et relecture 8
10. Le pont, et l'accusé qui attend la confirmation 9
11. Le temps de l'événement, les fenêtres et les retards 9
12. L'état, et l'alarme qui ne clignote pas 10
13. Livrer une seule fois, en effet 11
14. Lambda et Kappa 12
Points clés à retenir 13
Exercices 13

Cours 09 — Ingérer un flux d'événements

Théorie · Cours 9/12 — Formation Data Engineering · intermédiaire

Où vous êtes : source → **ingestion** → lac → entrepôt → restitution, sur le chemin chaud. Les huit premiers cours ont tout traité par lots : un fichier arrive, on le lit, on le range, on s'arrête. Celui-ci traite les messages un à un, pendant qu'ils arrivent, avec des programmes qui ne finissent pas. Le chemin est le même, source puis ingestion ; ce qui change, c'est qu'on n'attend plus que la journée soit finie pour la regarder.

En 30 secondes :

© 2026 iAlexMG. Tous droits réservés.

- Le temps réel se paie, en services à faire tourner et en erreurs d'un genre nouveau. Chez Boréal, il gagne une quinzaine d'heures sur une alerte qui a de toute façon six jours d'avance : c'est la question à poser avant de s'y lancer.
- MQTT est le protocole léger du terrain. Des éditeurs publient sur des sujets hiérarchiques, un courtier relaie, des abonnés reçoivent. Le courtier ne garde rien, sauf ce qu'on lui demande.
- La qualité de service 1 livre au moins une fois : sur une semaine de télémétrie, 70 messages arrivent en double, et 246 arrivent plus d'une heure après la fin de leur sortie.
- L'archive quotidienne de la télémétrie, que le chemin froid charge depuis le Cours 03, est ce qu'un abonné a écrit. Rejouée sur le courtier et réécoutée, elle revient identique, ligne pour ligne.
- Kafka, lui, garde tout, dans l'ordre, et chacun relit à son rythme. La clé d'un message décide de sa partition, et donc de l'ordre dans lequel on le lira.
- Une fenêtre se ferme sur le temps des événements, jamais sur l'horloge du poste. Sur la vibration du compresseur 2, une fenêtre de 15 minutes lève l'alerte le 22 octobre 2025 à 8 h 15, six jours et deux heures et demie avant le déclenchement.

L'idée du cours. Un flux n'est pas un fichier qu'on lirait plus vite. Un fichier est fini : on sait quand on a tout lu, et on peut compter. Un flux ne finit jamais, arrive dans le désordre, en double, parfois en retard de plusieurs jours, et il faut pourtant décider maintenant. Tout le cours tient dans cette tension : agir sur ce qu'on sait déjà, sans jamais faire semblant de tout savoir.

Le cours compte deux séances. La première, sections 1 à 6, tient MQTT : le terrain qui publie. La seconde, sections 7 à 14, tient Kafka : le flux qu'on garde, qu'on relit, et dont on tire une alerte.

Objectifs

- Dire ce que le temps réel apporte et ce qu'il coûte, et mesurer l'avance qu'il donne vraiment.
- Expliquer le rôle d'un courtier MQTT, d'un éditeur et d'un abonné, et lire un sujet avec ses jokers.
- Distinguer les trois qualités de service, et dire ce que « au moins une fois » oblige à faire en aval.
- Employer le message retenu et la session persistante, et connaître la borne de ce qu'un courtier garde.
- Reconnaître l'archive d'un flux comme ce qu'un abonné a écrit, et le prouver.
- Décrire Kafka comme un journal : sujets, partitions, décalages, rétention.
- Choisir la clé d'un message pour obtenir l'ordre dont le traitement a besoin.
- Faire relire un sujet par un groupe de consommateurs neuf, sans rien demander à la source.

© 2026 iAlexMG. Tous droits réservés.

- Écrire un pont qui n’acquitte un message qu’une fois qu’il est en lieu sûr.
- Calculer une fenêtre sur le temps des événements, et dire ce que deviennent les données en retard.
- Donner une alarme qui ne clignote pas, par une bande morte.
- Rendre une écriture idempotente, et en tirer une livraison effectivement unique.
- Situer Lambda et Kappa, et dire ce que chacune coûte.

---

1. Ce que le temps réel apporte, et ce qu’il coûte

Labo : `Labo_09_DE.md`, étape 14.

Un traitement en continu réagit à chaque message au lieu d’attendre la fin de la journée. Il en tire deux choses qu’un lot ne donne pas : une décision plus tôt, et une vue de ce qui se passe maintenant. Il le paie de trois façons : des services qui tournent jour et nuit et qu’il faut surveiller, des programmes qui gardent un état en mémoire et qu’on ne relance pas comme un script, et des erreurs d’un genre nouveau, les doublons, les retards, le désordre.

La question honnête se pose avant d’écrire la première ligne : à quelle vitesse la décision doit-elle vraiment être prise. La plupart des besoins qu’on dit « en temps réel » sont des besoins horaires, parfois quotidiens, et un DAG qui tourne toutes les heures y répond pour une fraction du coût.

Boréal donne un cas mesuré. Le compresseur d’air 2 dérive à partir du 15 octobre 2025 et déclenche le 28 à 10 h 44. Une fenêtre de 15 minutes sur sa vibration, jugée contre le seuil de la procédure d’entretien, lève l’alerte le 22 octobre à 8 h 15. Le chemin froid, qui tourne chaque nuit sur la veille depuis le Cours 06, aurait vu la même fenêtre le 23 au tout début de la journée : une quinzaine d’heures plus tard, et toujours plus de cinq jours avant la panne.

Ce qu’on compare	**Chez Boréal**
Première fenêtre au-dessus de 4,5 mm/s	le 22 octobre, de 8 h à 8 h 15
L’alerte en continu	le 22 octobre à 8 h 15, 6 jours 2 h 29 min avant le déclenchement
Le même calcul, lancé chaque nuit sur la veille	le 23 octobre peu après minuit, encore plus de 5 jours d’avance
Le bon de travail ouvert pour un bruit anormal	le 22 octobre à 9 h 12, fermé à 10 h 04 : « rien constaté »

Ce que le temps réel gagne ici, c’est la dernière ligne. À 9 h 12, un technicien écoute le compresseur et ne trouve rien ; l’alerte, elle, est levée depuis presque une heure, et lui aurait dit quoi chercher. Le gain ne se mesure donc pas en heures d’avance, mais en décisions qu’on prend pendant que l’information est encore utile. C’est ce critère-là qui justifie un chemin chaud, et il se vérifie cas par cas.

© 2026 iAlexMG. Tous droits réservés.

2. MQTT : un courtier, des éditeurs, des abonnés

Labo : `Labo_09_DE.md`, étape 1.

MQTT est un protocole de messagerie par publication et abonnement, né à la fin des années 1990 pour surveiller des oléoducs par des liaisons satellites lentes et chères. Il en a gardé ce qui fait son succès sur le terrain : un en-tête de deux octets, une connexion qui reste ouverte, et un client assez petit pour tenir dans un capteur.

Trois rôles, et aucun ne connaît les autres :

Rôle	Ce qu'il fait	Chez Boréal
L'éditeur	publie un message sur un sujet, sans savoir qui l'écoute	le module télématique d'un véhicule, la passerelle de l'usine
Le courtier	reçoit chaque message et le relaie à chaque abonné du sujet	Mosquitto, dans un conteneur de la plateforme
L'abonné	déclare les sujets qui l'intéressent, et reçoit ce qui s'y publie	l'abonné qui écrit l'archive de la télémétrie, puis le pont vers Kafka

Le découplage est le point fort. Un véhicule ne sait pas qu'un abonné écrit ses messages dans un fichier, ni qu'un pont les copie ailleurs : ajouter un consommateur ne touche à aucun éditeur. Le revers est dans la même phrase. Le courtier relaie, il ne conserve pas : un message publié quand personne n'écoute est perdu, sauf dans les deux cas de la section 5.

Le courtier de Boréal est Mosquitto 2.1.2, dans une image de 41,8 Mo qui occupe 2,3 Mo de mémoire une fois démarrée. Sans configuration, Mosquitto 2 n'accepte que les clients de sa propre machine, c'est-à-dire de l'intérieur de son conteneur ; le fichier `mosquitto.conf` de la plateforme ouvre le port 1883 et accepte les clients anonymes, ce qu'aucune installation hors d'un poste de formation ne devrait faire.

3. Les sujets, et leurs jokers

Labo : `Labo_09_DE.md`, étapes 1 et 2.

Un sujet est une chaîne découpée par des barres obliques, du plus général au plus précis. Il n'a pas à être déclaré : il existe dès qu'on y publie. Boréal en tient deux familles.

Sujet	Qui publie	Un message par
`boreal/usine/UTL/CMP2/VIBRATION`	la passerelle de l'usine	point de l'historien de l'étiquette `UTL.CMP2.VIBRATION`

© 2026 iAlexMG. Tous droits réservés.

Sujet	Qui publie	Un message par
`boreal/vehicules/MN26000406/sorties`	le module télématique du véhicule `MN26000406`	fin de sortie

L'usine reprend la hiérarchie des actifs du Cours 08, la zone puis l'équipement puis la grandeur, et c'est ce qui rend les jokers utiles. Un abonné les emploie pour écouter plusieurs sujets d'un coup.

Joker	Ce qu'il remplace	Exemple
`+`	exactement un niveau	`boreal/vehicules/+/sorties` : les sorties de tous les véhicules
`#`	tous les niveaux qui suivent, en dernière position seulement	`boreal/usine/UTL/CMP2/#` : les quatre points du compresseur 2

Le choix des niveaux est une décision de conception, et elle dure : chaque abonné écrit ses jokers en fonction d'elle. Un bon sujet va du plus stable au plus variable, et ne porte jamais ce qui change à chaque message, une date ou une valeur, qui ferait un sujet par message.

4. La qualité de service, et ce que « livrer » veut dire

Labo : `Labo_09_DE.md`, étapes 3 et 4.

Un message voyage en deux sauts, de l'éditeur au courtier puis du courtier à l'abonné, et chaque saut peut perdre ou répéter. MQTT laisse choisir, message par message, la garantie qu'on achète.

Qualité de service	Ce que le protocole fait	Ce que l'abonné peut recevoir
0	envoie une fois, sans accusé	le message, ou rien : au plus une fois
1	renvoie tant que l'accusé n'est pas revenu	le message, ou le message deux fois : au moins une fois
2	un échange en quatre temps qui écarte les doublons	le message, exactement une fois, au prix de deux allers-retours

La télémétrie de Boréal est publiée en qualité 1. Rien ne se perd, mais un message dont l'accusé s'est égaré en route est renvoyé, et l'abonné le reçoit deux fois, avec le même identifiant `message_id`. Sur la semaine du 10 au 16 janvier 2026, 3 449 lignes portent 3 379 messages : 70 doublons. C'est la règle, pas une panne.

La qualité 2 semble régler la question, et elle ne la règle qu'à moitié : elle garantit l'unicité entre deux clients MQTT, pas dans ce que l'abonné fait ensuite. Un abonné qui plante après avoir écrit le message et avant d'acquitter le recevra de nouveau à son retour. La seule garantie de bout en bout se construit en aval, par une écriture qui ne double rien (section 13). C'est pour cela que la qualité 1 suffit presque toujours : on accepte le doublon, et on s'organise pour qu'il soit sans effet.

© 2026 iAlexMG. Tous droits réservés.

Un détail de l'abonné compte ici. Le client Python, paho, acquitte un message de qualité 1 au retour de la fonction qui le traite, et pas avant. L'abonné du labo écrit donc chaque ligne sur le disque avant de rendre la main : un abonné tué en plein travail ne perd rien de ce qu'il a acquitté, et ce qu'il n'a pas acquitté reviendra.

5. Le message retenu, et la session persistante

Labo : `Labo_09_DE.md`, étapes 6 et 7.

Le courtier ne garde rien, disait la section 2, sauf dans deux cas qu'on lui demande explicitement.

Le message retenu d'abord. Un éditeur peut marquer un message comme retenu : le courtier garde le dernier de chaque sujet, et le donne aussitôt à tout nouvel abonné. C'est l'usage naturel d'un état, la dernière valeur connue. La passerelle de l'usine publie ainsi chaque point : un abonné qui arrive en cours de journée reçoit d'un coup la dernière valeur des 58 étiquettes, au lieu d'attendre que chacune bouge. Un message retenu n'est pas un message neuf ; il arrive marqué comme tel, et un abonné qui l'écrirait comme une réception compterait deux fois un point déjà vu.

La session persistante ensuite. Un abonné qui se connecte toujours sous le même identifiant et demande une session persistante (`clean_session` à faux, en MQTT 3.1.1) garde ses abonnements chez le courtier quand il se déconnecte. Pendant son absence, le courtier lui met de côté les messages de qualité 1 et 2, et les lui livre d'un coup à son retour.

Deux limites à connaître avant de s'y fier. La file que le courtier garde à un absent est bornée : Mosquitto la limite à 1 000 messages par défaut, et jette le reste sans rien dire. Mesuré chez Boréal, un abonné absent pendant que 2 118 messages passaient n'en a retrouvé que 999 ; la plateforme relève la borne à 100 000 dans `mosquitto.conf`. Et ce qui a attendu arrive en retard : après une coupure de l'abonné pendant le rejeu de la semaine de télémétrie, les 3 449 lignes sont bien là, mais celles qui ont attendu portent l'heure du retour : sur trois essais, jusqu'à 30 à 33 heures après leur réception d'origine, et de 245 à 297 changent de fichier quotidien. Une session persistante sauve les messages, pas leur heure.

Piège — une session persistante continue de remplir sa file. Le courtier garde tout pour un abonné absent, tant que celui-ci ne revient pas en session propre. Un abonné du labo oublié à la fin d'une séance reçoit, à la séance suivante, tout ce qui s'est publié entre les deux : en préparant ce cours, un abonné rouvert a reçu d'un bloc 62 726 points et 3 449 résumés qui dataient d'essais antérieurs. Une session qu'on n'utilise plus se ferme par une connexion en session propre.

MQTT 5 remplace ce drapeau par une durée d'expiration de la session, que le client choisit. Le labo emploie MQTT 3.1.1, que tous les courtiers et tous les clients parlent.

6. L'archive, c'est ce qu'écrit un abonné

Labo : `Labo_09_DE.md`, étapes 3, 4 et 5.

Depuis le Cours 01, la télémétrie arrive dans `sources/telemetrie/`, un fichier JSON lines par jour, et le Cours 03 la charge dans la `RAW`. Ce fichier n'est pas une source au sens où l'ERP en est une. Sa note, `LISEZMOI.md`, le dit en une ligne : c'est ce qu'écrit l'abonné du courtier. Chaque ligne porte le sujet, l'heure de réception, la qualité de service et le message ; les doublons y sont, les retards aussi, rangés dans le fichier du jour où ils sont arrivés.

© 2026 iAlexMG. Tous droits réservés.

Le chemin froid et le chemin chaud lisent donc le même flux à deux moments. Le chaud le voit passer, message par message ; le froid ramasse le lendemain ce qu'un abonné en a gardé. Rejouer une semaine de l'archive sur le courtier et la réécouter avec l'abonné du labo le prouve : 3 449 lignes de part et d'autre, les mêmes jour par jour, et aucune différence de contenu dans un sens ni dans l'autre. Seule l'heure de réception bouge, de quelques secondes, parce que le rejeu en temps accéléré ne tombe pas à la milliseconde.

Cette identité a une conséquence qui dépasse Boréal. Tout ce que le chemin froid sait de la télémétrie, il le tient d'un abonné. Si l'abonné tombe, l'archive a un trou, et aucun traitement en aval ne le comblera : le premier maillon d'un chemin froid est souvent un programme en continu.

7. Kafka : un journal qu'on garde

Labo : `Labo_09_DE.md`, étapes 9 et 15.

MQTT relaie ; Kafka conserve. Un sujet Kafka est un journal : une suite de messages qu'on ne fait qu'ajouter au bout, chacun à une position numérotée, son décalage (*offset*). Personne ne retire un message en le lisant. Un lecteur tient seulement sa position, et avance à son rythme.

Notion	Ce que c'est	Chez Boréal
Sujet (*topic*)	un journal nommé	`usine.points`, `vehicules.sorties`, `usine.alertes`
Partition	un morceau du sujet, avec son propre ordre et ses propres décalages	trois pour `usine.points`, une pour `usine.alertes`
Décalage	la position d'un message dans sa partition, qui ne change jamais	de 0 à 24 811 dans la partition 0 d'`usine.points`, après le rejeu d'octobre
Rétention	combien de temps, ou d'octets, le journal garde	sept jours par défaut, sans borne de taille

La rétention change tout. Tant qu'un message est dans le journal, n'importe quel lecteur peut le relire, aujourd'hui ou demain, et un lecteur neuf peut repartir du début. C'est la différence de fond avec MQTT : un abonné MQTT qui arrive en retard n'a que les messages retenus, un par sujet ; un consommateur Kafka qui arrive en retard a tout ce que la rétention garde.

Kafka tourne ici en mode KRaft, le mode où Kafka tient lui-même la liste de ses sujets et de ses partitions, par son propre protocole de consensus, sans ZooKeeper. Depuis Kafka 4.0, c'est le seul mode. La plateforme en fait tourner un seul nœud, à la fois courtier et contrôleur, qui occupe environ 290 Mo de mémoire au repos et répond moins de cinq secondes après son démarrage. Un seul nœud veut dire aucune copie des partitions, ce qu'aucune production n'accepterait : le facteur de réplication de chaque sujet vaut 1, et le cours le dit chaque fois qu'il crée un sujet.

Snowflake. Snowpipe Streaming charge des lignes directement dans une table, par un kit de développement ou une API REST, sans passer par des fichiers déposés dans un stage ; sa documentation annonce une donnée interrogeable en 5 secondes au mieux. Un connecteur Kafka de Snowflake alimente les tables depuis un sujet. Côté transformation, un stream enregistre les changements d'une table depuis sa dernière lecture, et une tâche les traite dès qu'il en porte. D'après la documentation de Snowflake, consultée le 2026-09-23.

© 2026 iAlexMG. Tous droits réservés.

Databricks. Structured Streaming traite un flux comme une table qui grandit, avec les mêmes notions de fenêtre, de filigrane et d'état que ce cours. Une limite compte pour le lecteur de Free Edition, qui ne fournit que du calcul serverless : sur ce calcul, le déclencheur recommandé est `AvailableNow`, qui traite ce qui est arrivé puis s'arrête, et les déclencheurs continu ou à intervalle fixe se refusent. D'après la documentation de Databricks, consultée le 2026-09-23.

8. Partitions, clés et ordre

Labo : `Labo_09_DE.md`, étapes 9 et 10.

Un sujet se coupe en partitions pour qu'on puisse l'écrire et le lire à plusieurs. Le prix est dans une phrase qu'il faut savoir par cœur : Kafka ne garantit l'ordre qu'à l'intérieur d'une partition. Deux messages de deux partitions différentes n'ont entre eux aucun ordre garanti.

Le producteur choisit la partition d'un message d'après sa clé : la même clé tombe toujours dans la même partition. La clé est donc l'outil qui décide de l'ordre, et on la choisit en fonction de ce que le traitement devra lire ensemble.

Choix de clé pour `usine.points`	**Ce qu'il donne**
aucune clé	les messages se répartissent à peu près au hasard ; l'état de marche et la vibration d'un même compresseur peuvent se lire dans le désordre
l'étiquette, `UTL.CMP2.VIBRATION`	chaque étiquette dans l'ordre, mais pas les étiquettes d'un même équipement entre elles
l'équipement, `UTL-CMP2`	les quatre points du compresseur 2 dans l'ordre où ils se sont produits

Boréal clé par équipement, parce que l'alerte de vibration juge la vibration pendant la marche : il lui faut la marche et la vibration du même compresseur dans l'ordre. Les dix-sept équipements se répartissent entre trois partitions, de tailles inégales : après le rejeu d'octobre, 24 812, 21 765 et 16 149 messages. Le compresseur 2 partage la sienne avec le compresseur 1, le réseau d'air, le four et le sécheur.

La clé ne s'improvise pas après coup. Changer de clé, ou de nombre de partitions, change la partition des messages suivants sans toucher aux anciens : l'ordre d'une clé n'est plus garanti à la frontière.

9. Groupes de consommateurs, décalages et relecture

Labo : `Labo_09_DE.md`, étapes 11 et 15.

Un consommateur lit au nom d'un groupe. Kafka répartit les partitions d'un sujet entre les membres du groupe, une partition n'étant lue que par un seul membre à la fois, et retient pour le groupe le décalage du prochain message à lire. Deux conséquences.

Un groupe se partage le travail. Trois consommateurs du même groupe sur un sujet de trois partitions lisent chacun la sienne ; un quatrième attendrait sans rien faire. Le nombre de partitions borne donc le parallélisme d'un groupe.

© 2026 iAlexMG. Tous droits réservés.

Deux groupes ne se voient pas. Chacun a ses propres décalages, et lit tout le sujet de son côté. Chez Boréal, le groupe `vers-series` écrit la base de séries temporelles, le groupe `alerte-vibration` calcule l'alerte, et ni l'un ni l'autre ne ralentit l'autre. Un groupe neuf commence où on le lui dit : au plus ancien message gardé, avec `auto.offset.reset` à `earliest`.

C'est ce qui rend la relecture si simple. Un second groupe d'alerte, avec un seuil de 4,0 mm/s au lieu de 4,5, relit les 62 726 points d'octobre en moins d'une seconde, et lève son alerte le 21 octobre à 6 h 45, un jour plus tôt, sans rien demander au courtier MQTT ni à l'usine. Avec MQTT seul, la même question aurait demandé de rejouer toute la source.

Le moment où un groupe enregistre son décalage est une décision, et la section 13 y revient : un décalage enregistré trop tôt perd des messages, trop tard en fait relire.

10. Le pont, et l'accusé qui attend la confirmation

Labo : `Labo_09_DE.md`, étapes 10 et 13.

Le terrain parle MQTT ; ce qui se garde et se relit vit dans Kafka. Entre les deux, un pont s'abonne au courtier et écrit chaque message dans un sujet Kafka. Le pont de Boréal est un programme Python d'une centaine de lignes, avec un client de chaque côté, et deux choix y font tout.

Le premier est la clé, vue à la section 8. Le second est le moment de l'accusé. Le pont reçoit un message en qualité 1 : tant qu'il ne l'a pas acquitté, le courtier le lui relivrera. Il ne doit donc l'acquitter qu'une fois le message en lieu sûr, c'est-à-dire écrit par Kafka. L'accusé ne part pas au retour de la fonction qui reçoit, comme chez l'abonné, mais du rappel de livraison du producteur Kafka, quand Kafka confirme. Un pont qui plante entre les deux n'a rien acquitté : le courtier relivrera, Kafka recevra peut-être un doublon, et la section 13 dit pourquoi ce n'est pas grave.

Il y a une façon plus simple d'obtenir la même garantie : attendre la confirmation de chaque message avant de passer au suivant. Elle est correcte, et elle s'est révélée inutilisable sur le poste de l'auteur. Chaque confirmation y coûte 44 ms dès que le message dépasse 200 octets environ, contre 0,5 ms en dessous ; le même client lancé depuis Linux répond en 0,4 ms à toutes les tailles. C'est le relais qui mène des programmes de Windows aux ports de WSL 2 qui retient ces envois. Un pont qui attend chaque confirmation plafonne alors à 22 messages par seconde, sous le débit d'une fin de semaine de télémétrie rejouée à 3 600 fois. Avec le rappel, le producteur groupe les messages en attente dans une même requête, et le courtier en laisse vingt en vol : le pont suit sans peine.

Pourquoi — un pont écrit à la main, et non Kafka Connect. Kafka Connect est le cadre de Kafka pour brancher des sources et des cibles par configuration, sans code. Un connecteur MQTT y ferait le travail du pont. Celui de Confluent est commercial : sa documentation, consultée le 2026-09-23, le réserve à un essai de 30 jours, à une licence de développement sur un seul nœud, ou à une licence d'entreprise. D'autres existent en code ouvert. Le cours écrit le pont pour voir la clé et l'accusé, qu'un connecteur cacherait dans sa configuration ; en entreprise, Connect est souvent le bon choix, pour les mêmes raisons qui font préférer une bibliothèque à du code maison.

11. Le temps de l'événement, les fenêtres et les retards

Labo : `Labo_09_DE.md`, étapes 12 et 16.

© 2026 iAlexMG. Tous droits réservés.

Un message porte deux heures. Celle de l'événement, quand la chose s'est produite : l'instant d'un point, la fin d'une sortie. Et celle du traitement, quand le programme le voit. Elles diffèrent toujours un peu, parfois beaucoup : un résumé de sortie de la semaine rejouée arrive jusqu'à quatre jours après la fin de la sortie qu'il décrit, parce que le véhicule était hors de la couverture cellulaire.

Un calcul par fenêtre doit choisir son heure. Au temps de traitement, une fenêtre contient ce qui est arrivé pendant qu'elle était ouverte ; au temps de l'événement, ce qui s'est produit pendant sa durée. Seul le second a un sens pour une moyenne de vibration, et le rejeu accéléré le rend évident : à 3 600 fois, quinze minutes du poste contiennent quinze jours de Boréal.

Forme de fenêtre	**Ce qu'elle découpe**	**Un usage chez Boréal**
Fixe	des tranches qui se suivent sans se chevaucher : 8 h 00 à 8 h 15, puis 8 h 15 à 8 h 30	la vibration moyenne d'un compresseur, par quart d'heure
Glissante	une durée qui avance par pas plus courts : la dernière heure, recalculée toutes les 15 minutes	une tendance qu'on veut lisser sans attendre la fin de l'heure
De session	une activité, close par un silence : tout ce qu'un véhicule publie jusqu'à une heure sans message	les sorties d'une journée de motoneige

Reste à savoir quand fermer une fenêtre au temps de l'événement, puisqu'un message en retard peut toujours arriver. On se donne un filigrane (*watermark*) : l'affirmation « je ne recevrai plus rien d'antérieur à tel instant ». Une fenêtre se ferme quand le filigrane dépasse sa fin ; ce qui arrive ensuite est en retard, et il faut décider quoi en faire : l'écarter en le comptant, le verser à part, ou rouvrir la fenêtre et corriger le résultat déjà émis.

La semaine de télémétrie chiffre le choix, sur l'archive elle-même. En fenêtres de 15 minutes sur la fin des sorties, 306 des 3 379 messages arrivent après la fin de leur fenêtre. Un filigrane d'une heure en rattrape 65 ; les 241 autres arrivent plus tard encore, et un moteur qui ferme ses fenêtres une heure après leur fin les écarte. Et 18 sorties terminées avant le 10 janvier ne sont reçues que pendant la semaine : toute fenêtre qu'on aurait fermée pour les jours précédents était déjà fausse. Rejouée, la même semaine donne un peu plus de retardataires, de 369 à 374 sur trois rejeux : le rejeu date chaque réception de quelques secondes plus tard que l'archive, et ce qui arrivait juste avant la fin d'une fenêtre passe de l'autre côté. Ceux du filigrane bougent à peine, 241 ou 242.

Pour les points de l'usine, la question se pose moins. Le rejeu les publie dans l'ordre, la clé par équipement les garde dans l'ordre jusqu'à l'alerte, et une fenêtre se ferme simplement quand arrive un point plus récent que sa fin : le filigrane est le dernier instant vu. Le labo compte les points qui arriveraient malgré tout en retard sur ce filigrane : aucun, sur octobre.

12. L'état, et l'alarme qui ne clignote pas

Labo : `Labo_09_DE.md`, étapes 12, 13 et 14.

Un traitement en continu qui calcule une moyenne pondérée ne peut pas oublier chaque message après l'avoir lu. La moyenne d'une fenêtre, au Cours 08, pondère chaque valeur par la durée pendant laquelle elle a tenu, et la valeur qui tient au début d'une fenêtre est arrivée avant elle. Le programme garde donc,

© 2026 iAlexMG. Tous droits réservés.

pour chaque compresseur, un état : la dernière vibration, le dernier état de marche, l'instant du dernier point, et les sommes des fenêtres encore ouvertes. Cet état vit en mémoire, et c'est la vraie différence avec un script : un programme à état qu'on redémarre doit le reconstruire, en relisant ce qu'il faut du journal.

L'alerte de Boréal juge chaque fenêtre de 15 minutes qui porte au moins dix minutes de marche, contre les deux seuils de la procédure d'entretien du compresseur, publiée dans le corpus : alerte à 4,5 mm/s, arrêt immédiat à 7,1 mm/s.

Changement d'état	Quand	Moyenne de la fenêtre
Alerte levée	le 22 octobre 2025 à 8 h 15	4,545 mm/s, de 8 h à 8 h 15
Arrêt levé	le 27 octobre à 6 h 15	7,554 mm/s, de 6 h à 6 h 15
Déclenchement du compresseur	le 28 octobre à 10 h 44	le bon `BT-2025-00742` s'ouvre à 10 h 50
Alerte et arrêt retombés	le 30 octobre à 14 h	2,503 mm/s, après le remplacement de la soupape et du roulement

Le premier jour de la dérive, la moyenne oscille autour du seuil. Une alarme qui se lève dès qu'elle le dépasse et retombe dès qu'elle passe dessous se lève six fois le 22 octobre, et une alarme qui clignote finit par ne plus être lue. La parade est la même que celle de l'historien au Cours 08 : une bande morte. L'alerte se lève à 4,5 mm/s et ne retombe que sous 4,0 ; elle se lève alors une seule fois, et retombe après la réparation. La même règle, sur les compresseurs 1 et 3, ne lève rien en 182 jours.

À retenir — la logique d'un programme en continu se teste sans le flux. Les fenêtres et l'alarme du labo vivent dans un module sans aucune dépendance, que trois tests vérifient en une fraction de seconde, sur des points écrits à la main. Kafka, le courtier et la base ne sont que la plomberie autour. Un calcul de fenêtre qu'on ne peut vérifier qu'en rejouant une semaine est un calcul qu'on ne vérifiera pas.

13. Livrer une seule fois, en effet

Labo : `Labo_09_DE.md`, étapes 11 et 16.

Chaque maillon du chemin chaud de Boréal livre au moins une fois : le courtier en qualité 1, le pont qui n'acquitte qu'après Kafka, le consommateur qui relit un lot après une panne. Aucun ne garantit l'unicité, et il serait coûteux de l'exiger de chacun. On l'obtient à la fin de la chaîne, par deux règles.

La première : l'écriture est idempotente. Chaque table du schéma `temps_reel` a une clé primaire qui identifie le fait, l'étiquette et l'instant pour un point, l'identifiant du message pour une sortie, et chaque écriture passe par `INSERT … ON CONFLICT DO NOTHING`. Écrire deux fois le même fait ne change rien. Sur la semaine de télémétrie, le consommateur lit 3 449 messages et en insère 3 379 : les 70 doublons de la qualité 1 s'arrêtent là, sans une ligne de code qui les cherche.

La seconde : l'ordre des validations. Le consommateur valide son lot dans la base, puis seulement son décalage dans Kafka. Une panne entre les deux fait relire le lot au redémarrage, et la clé primaire l'écarte. L'ordre inverse perdrait le lot : Kafka le croirait écrit, la base ne l'aurait pas.

© 2026 iAlexMG. Tous droits réservés.

Garantie	Ce qu'elle promet	Comment Boréal l'obtient
Au plus une fois	rien n'est doublé, quelque chose peut manquer	décalage enregistré avant le traitement : jamais employé ici
Au moins une fois	rien ne manque, quelque chose peut être doublé	accusé après la mise en lieu sûr, à chaque maillon
Exactement une fois, en effet	chaque fait se retrouve une fois dans la cible	au moins une fois, plus une écriture idempotente

Kafka offre aussi des transactions, qui rendent atomiques l'écriture de plusieurs messages et l'enregistrement d'un décalage. Elles servent quand la cible est Kafka lui-même ; pour une base extérieure, l'écriture idempotente fait le même travail, plus simplement.

14. Lambda et Kappa

Labo : `Labo_09_DE.md`, étape 16.

Deux architectures se disputent la place du chemin chaud.

Lambda garde deux chemins séparés. Le chemin froid recalcule la vérité à partir de tout ce qui a été gardé, par lots, en prenant son temps ; le chemin chaud donne vite une réponse approchée, que le froid remplace ensuite. C'est exactement ce que Boréal fait : l'alerte est calculée en continu, et l'archive de la télémétrie, ramassée le lendemain, rattrape les 241 messages que le filigrane d'une heure a laissés de côté. Le prix est connu : deux programmes qui calculent la même chose de deux façons, et qui doivent rendre le même chiffre.

Kappa ne garde que le flux. Un seul programme, en continu ; pour recalculer, on relit le journal depuis le début avec une nouvelle version, comme le second groupe d'alerte de la section 9. Le prix est ailleurs : il faut que le journal garde assez longtemps. Kafka garde sept jours par défaut ; relire six mois demande de régler la rétention en conséquence, et d'en payer le disque.

	Lambda	Kappa
Programmes	deux, un par chemin	un seul
Recalculer	relancer le lot sur l'archive	relire le journal avec la nouvelle version
Ce qui coûte	tenir deux logiques d'accord	garder le journal assez longtemps
Chez Boréal	l'alerte en continu, la `RAW` et les séries par lots	la relecture d'octobre par un second groupe

Aucune n'est la bonne en soi. Le choix se fait au Cours 01 de la formation Architecture et Gouvernance, avec les autres choix de la pile ; ce cours en montre les deux mécanismes sur le même flux.

Recoupements — ce que ce cours partage avec ses voisines.

© 2026 iAlexMG. Tous droits réservés.

- Business Intelligence : le chemin chaud finit dans Grafana, branché au Cours 12 sur le schéma `temps_reel` que ce cours remplit. La conception du tableau de bord et sa lecture appartiennent à BI.
- Architecture et Gouvernance : Lambda et Kappa reviennent au Cours 01 d'Archi, comme choix d'architecture, avec leurs coûts d'exploitation. Ici, on en voit le mécanisme ; là-bas, on en décide.

Points clés à retenir

- Le temps réel se justifie par une décision qu'on prend plus tôt, pas par une avance en heures. Chez Boréal, il gagne une quinzaine d'heures sur six jours, et c'est l'heure du bon de travail « rien constaté » qui en fait la valeur.
- MQTT découple l'éditeur de l'abonné par un courtier qui relaie sans garder. Un sujet va du plus stable au plus variable ; `+` remplace un niveau, `#` tous les suivants.
- La qualité 1 livre au moins une fois : les doublons sont la règle. On les rend sans effet en aval plutôt que de les chasser en amont.
- Le message retenu donne la dernière valeur d'un sujet à tout nouvel abonné. La session persistante garde les messages d'un absent, dans une file bornée, et les livre en retard.
- L'archive d'un flux est ce qu'un abonné a écrit. La rejouer et la réécouter la redonne, ligne pour ligne.
- Kafka est un journal : on ajoute au bout, on lit à sa position, on garde selon une rétention. Un groupe neuf relit tout, sans rien demander à la source.
- L'ordre n'existe qu'à l'intérieur d'une partition, et la clé choisit la partition. On clé selon ce que le traitement doit lire ensemble.
- Un pont n'acquitte qu'une fois le message en lieu sûr, et laisse le producteur grouper ses envois.
- Une fenêtre se calcule au temps de l'événement et se ferme sur un filigrane ; ce qui arrive après est en retard, et se compte.
- Une alarme a une bande morte, comme un historien.
- Au moins une fois, plus une écriture idempotente, font une livraison effectivement unique. La base se valide avant le décalage, jamais après.
- Lambda recalcule par lots ce que le flux a approché ; Kappa relit le journal. Chacune paie son prix, en logique double ou en rétention.

Exercices

Les énoncés et leur corrigé sont dans `Exercices_corriges_09_DE.md`. Faites le labo avant : plusieurs questions portent sur des chiffres que vous aurez mesurés vous-même.

© 2026 iAlexMG. Tous droits réservés.